



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»**

ФАКУЛЬТЕТ

ИУ «Информатика и системы управления»

КАФЕДРА

ИУ-1 «Системы автоматического управления»

ОТЧЕТ

По лабораторной работе №3

«Исследование методов циклического кодирования»

По дисциплине

«Телекоммуникационные технологии»

Выполнил: Шевченко Алексей

Группа: ИУ1-42Б

Проверил: Лысухо Г. В.

Работа выполнена: 19.05.2023

Отчет сдан: 24.05.2023

Оценка:

Москва 2023

Цель работы

Реализация метода побитного циклического кодирования для заданной битовой последовательности.

Порядок выполнения лабораторной работы

- 1) Подготовка к выполнению лабораторной работы.
- 2) Реализация побитного циклического кодирования.
- 3) Реализация побитного “исключающего или”.

Подготовка к выполнению лабораторной работы

Был создан скрипт `lab_checksum.m`, в котором реализовывались все последующие задания. Кроме этого мы ознакомились со следующими разделами документации MathWorks MATLAB: `comm.CRCGenerator System object` и `comm.CRCDetector System object`; `Generate CRC code bits and append to input data`.

Реализация побитного циклического кодирования

Был задан кодовый полином CRC-4-ITU x^4+x+1 в побитном варианте и сформирован объект кодировщика и декодировщика циклического кода для заданного кодового полинома. Кроме этого было задано в десятичной системе число 46, осуществлён его перевод в двоичную систему, с помощью функцию `de2bi()` в 8-битном представлении. Далее исходное сообщение было закодировано заданным кодовым полиномом с помощью функции `step()`. Была вычислена длина полученного сообщения, длина закодированного сообщения и величина избыточности. Затем была смоделирована ошибка в пятом младшем бите закодированного по каналу связи сообщения и реализовано декодирование полученного сообщения. В конце было

реализовано вычисление синдрома ошибки и формирование таблицы синдромов ошибок для заданного кода. Код скрипта представлен на рисунках 1-2. Результат выполнения программы показан на рисунке 3.

```
clc
clear variables
close all
polynom = [1, 0, 0, 1, 1];
crcgenerator = comm.CRCGenerator(polynom, "ChecksumsPerFrame", 1);
crcdetector = comm.CRCDetector(polynom, "ChecksumsPerFrame", 1);
number_tmp = de2bi(46, 8, "left-msb");
k = length(number_tmp);
number_encoded = step(crcgenerator, number_tmp);
n = length(number_encoded);
number = zeros(n, 1);
number(1:k)=number_tmp;
izb = n-k;
error_decimal = 32;
error_binary = de2bi(error_decimal, length(number_encoded), "left-msb");
number_encoded_with_error = bitxor(number_encoded', error_binary);
decoded_with_error = step(crcdetector, number_encoded_with_error');
disp(['Порождающий полином g(x): ', num2str(polynom)]);
disp(['Исходное сообщение m(x): ', num2str(number)']);
disp(['Исходное сообщение m(x) (в десятичной): ', num2str(46)]);
[~, r]=gfdeconv(wrev(number'),wrev(polynom));
ostdel=wrev(r);
disp(['Остаток от деления исходного сообщения m*(x) на порождающий полином g(x): ', num2str(ostdel)]);
disp(['Закодированное сообщение c(x): ', num2str(number_encoded)']);
disp(['Модель ошибки e(x): ', num2str(error_binary)']);
disp(['Полученное сообщение r(x): ', num2str(number_encoded_with_error)']);
disp(['Декодированное сообщение r(x)/g(x): ', num2str(decoded_with_error)']);
s_x_2 = zeros(n, izb);
error_decimal = 1;
```

Рис. 1. Код скрипта в MATLAB, который реализует побитное циклическое кодирование

```

error_decimal = 1;
e_x = cell(n, 1);
s_x = cell(n, 1);
for i=1:n
    error_decimal=2^(i-1);
    error_binary = de2bi(error_decimal, length(number_encoded), "left-msb");
    number_encoded_with_error = bitxor(number_encoded', error_binary);
    [~, syndrome]=gfdeconv(wrev(number_encoded_with_error),wrev(polynom));
    if length(syndrome) < izb
        syndrome(length(syndrome)+1:izb)=0;
    end
    s_x_2(i, :) = wrev(syndrome);
    e_x{i} = strcat('x^', num2str(i-1));
    str='';
    cnt = 0;
    for j=1:izb
        if s_x_2(i, j) ~= 0
            cnt = cnt+1;
            cnt_limit=sum(s_x_2(i, :));
            if cnt~=cnt_limit
                str = strcat(str, string(strcat('x^', num2str(izb-j), '+')));
            else
                str = strcat(str+string(strcat('x^', num2str(izb-j))));
            end
            s_x{i}=str;
        end
    end
end
table(e_x, s_x, s_x_2)

```

Рис. 2. Код скрипта в MATLAB, который реализует побитное циклическое кодирование

Порождающий полином $g(x)$: 1 0 0 1 1
 Исходное сообщение $m(x)$: 0 0 1 0 1 1 1 0 0 0 0 0
 Исходное сообщение $m(x)$ (в десятичной): 46
 Остаток от деления исходного сообщения $m(x)$ на порождающий полином $g(x)$: 1 0 1 1
 Закодированное сообщение $c(x)$: 0 0 1 0 1 0 1 1 0 1 0 1 1
 Модель ошибки $e(x)$: 0 0 0 0 0 0 0 1 0 0 0 0 0
 Полученное сообщение $r(x)$: 0 0 1 0 1 1 0 0 1 0 1 1
 Декодированное сообщение $r(x)/g(x)$: 0 0 1 0 1 1 0 0

 ans =
 12×3 [table](#)

e_x	s_x	s_x_2			
{ 'x^0' }	{ ["x^0"]}	0	0	0	1
{ 'x^1' }	{ ["x^1"]}	0	0	1	0
{ 'x^2' }	{ ["x^2"]}	0	1	0	0
{ 'x^3' }	{ ["x^3"]}	1	0	0	0
{ 'x^4' }	{ ["x^1+x^0"]}	0	0	1	1
{ 'x^5' }	{ ["x^2+x^1"]}	0	1	1	0
{ 'x^6' }	{ ["x^3+x^2"]}	1	1	0	0
{ 'x^7' }	{ ["x^3+x^1+x^0"]}	1	0	1	1
{ 'x^8' }	{ ["x^2+x^0"]}	0	1	0	1
{ 'x^9' }	{ ["x^3+x^1"]}	1	0	1	0
{ 'x^10' }	{ ["x^2+x^1+x^0"]}	0	1	1	1
{ 'x^11' }	{ ["x^3+x^2+x^1"]}	1	1	1	0

Рис. 3. Результат выполнения программы

Реализация побитного “исключающего или”

В качестве исходного сообщения использовались 7 младших бит десятичного числа 46. Было реализовано вычисление бита четности “исключающим или”, который записан в старший бит итогового 8-битного закодированного сообщения. Далее было необходимо сначала смоделировать ошибку в пятом младшем бите, а затем одновременно в третьем и пятом младших битах. В конце была осуществлена проверка достоверности передачи в обоих случаях. Код скрипта представлен на рисунке 4. Результат выполнения программы показан на рисунке 5.

```
p = 1;
for i=2:length(number_tmp)
    p=bitxor(number_tmp(i), p);
end

number_tmp(1)=p;
disp(['Исходное сообщение с битом чётности в старшем бите: ', num2str(number_tmp')]);

crcdetector = comm.CRCDetector(polynom, "ChecksumsPerFrame", 1);
number_encoded_with_error=bitxor(number_encoded', error_binary);
[~, error] = step(crcdetector, number_encoded_with_error');

disp(['Проверка достоверности передачи при 1-й ошибке (1-есть ошибка, 0-нет ошибки): ', num2str(error)]);

errors_binary=2^3+2^5;
number_encoded_with_errors=bitxor(number_encoded', errors_binary);
[~, error] = step(crcdetector, number_encoded_with_errors');
disp(['Проверка достоверности передачи при 2-х ошибках (1-есть ошибка, 0-нет ошибки): ', num2str(error)]);
```

Рис. 4. Код скрипта в MATLAB, который реализует побитное “исключающего или”

```
Исходное сообщение с битом чётности в старшем бите: 1 0 1 0 1 1 1 0
Проверка достоверности передачи при 1-й ошибке (1-есть ошибка, 0-нет ошибки): 1
Проверка достоверности передачи при 2-х ошибках (1-есть ошибка, 0-нет ошибки): 0
```

Рис. 5. Результат выполнения программы

Вывод

В первом случае, при одной ошибке, использование побитного “исключающего или” говорит нам об ошибке, однако, во втором случае, при двух ошибках, алгоритм показывает, что ошибок нет. Это явный пример недостатка контрольной суммы “исключающим или”. При четном количестве

ошибок алгоритм этого не покажет. Следовательно, можно сделать вывод, что такой алгоритм лучше не использовать, так как он достаточно неточный.